

Project 3: CS 491

Program 1: Parser

CFG Analysis

The program follows a sequence of character checks forming a state machine. The execution path requires specific characters in order:

Each conditional branch checks for a specific char, meaning the correct input must match this exact sequence to reach the crash condition.

Based on the branch condition, the following tokens were extracted:

"DOC" "ATT" "<" ">"

These tokens were added to a dictionary file to guide AFL++ toward meaningful inputs

Results Comparison:

Metric	Without Dictionary	W/Dictionary
Time to First Crash	57 sec	9 sec
Saved crashes	1	3
Total Crashes	73	66
Corpus Count	8	6
Levels Reached	6	4
Total Executions	553k	26.1k

Analysis

The CFG guided dictionary dramatically improves fuzzing performance. With the correct tokens, AFL++ reaches the crash in seconds (9 seconds). Without the dictionary, it takes significantly longer to reach the crash compared to the dictionary approach. Additionally, the number of executions required is reduced.

Program 2: Mathematical Constraint (Checksum)

CFG Analysis

The program enforces an arithmetic condition where the sum of four bytes must equal 400 and the first and last bytes must match. This constraint depends on numerical relationships rather than tokens.

Results

Metric	Without Dictionary	W/Dictionary
Time to First Crash	1 min 15 secs	28
Saved crashes	1	1
Total Crashes	137	74
Corpus Count	2	2
Levels Reached	2	2
Total Executions	187k	69.4k

Analysis

The checksum-based parser relies on satisfying an arithmetic constraint, along with additional equality condition between two bytes. Unlike the first program, this logic does not depend on fixed string tokens but instead on numeric relationships. A CFG guided dictionary consisting of high value byte constants was used to assist. While the dictionary slightly improved the time to first crash and reduced the number of executions needed, the overall impact was limited. This demonstrated dictionary-based fuzzing is less effective for programs such as this.

Program 3: Magic Parser

CFG Analysis

The program checks for specific string patterns using branch conditions. Execution requires the input the being with “MAG” and later contain “END” to reach the crash condition. These comparisons from key control flow paths must be satisfied.

Based on these conditions, the following tokens were extracted “MAG” “END” These tokens were added to the dictionary to guide AFL++ toward valid structured inputs.

Metric	Without Dictionary	W/Dictionary
Time to First Crash	None	39 sec
Saved crashes	0	1
Total Crashes	0	134
Corpus Count	2	7
Levels Reached	2	6
Total Executions	152k	109k

Analysis

The CFG guided dictionary significantly improved fuzzing performance. Without the dictionary, AFL++ was unable to generate inputs that satisfied the required string conditions, resulting in no crashes. With the dictionary, AFL++ quickly produced valid input matching “MAG” and “END”, allowing it to explore deeper paths and trigger crashes.

This demonstrates that dictionary-based fuzzing is highly effective for programs that rely on structured input and fixed comparisons.